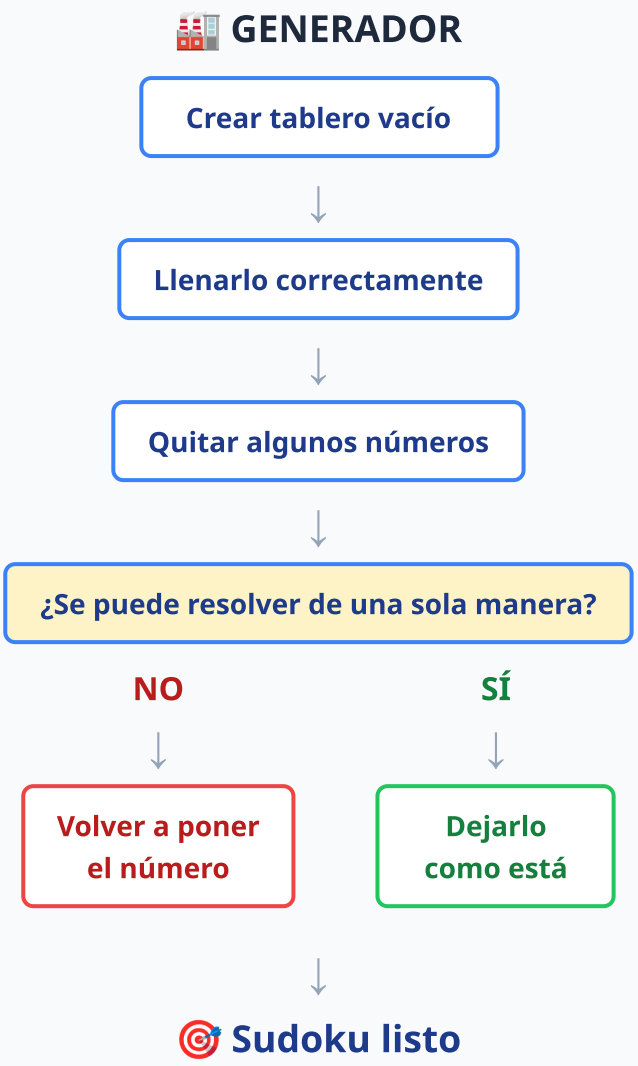
Nuestro motor de Sudoku hasta ahora

Pensalo como si tuviéramos una máquina que fabrica Sudokus paso a paso.



Ahora veamos qué hace cada función en detalle.

1. crearTableroVacio()

Esta función simplemente crea un Sudoku vacío.

```
function crearTableroVacio() { return Array.from( { length: 9 }, () => Array(9).fill(0) ); }
```

Nos da un tablero donde el `0` significa: *"Acá todavía no hay ningún número."*

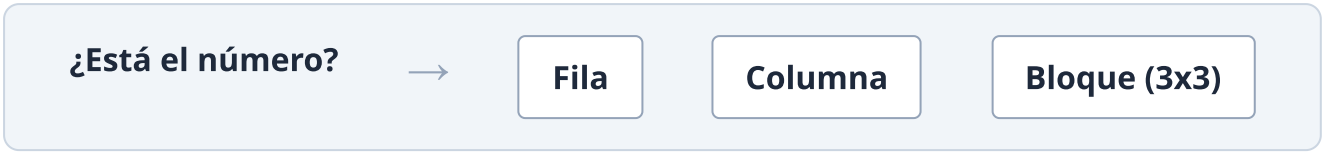
3. esValido()

Esta es nuestra regla principal del Sudoku. Le preguntamos a la función si un número específico puede ir en una posición.

```
esValido(tablero, fila, columna, numero)
```

Por ejemplo, `esValido(tablero, 5, 8, 9)` significa: *"¿Puedo poner un 9 en la fila 5, columna 8?"*

La función revisa que el número no esté repetido en tres lugares clave:



Si aparece en alguno: ❌ **No puedo ponerlo**.
Si no aparece en ninguno: ✅ **Puedo ponerlo**.

4. mezclar()

Esta función hace que los números aparezcan en un orden diferente. Por ejemplo, en lugar de intentar siempre en orden `[1, 2, 3, 4, 5, 6, 7, 8, 9]`, podemos obtener aleatoriamente `[7, 2, 9, 4, 1, 8, 5, 3, 6]`.

¿Por qué hacemos esto? Porque queremos que el generador no haga siempre el mismo Sudoku. Al mezclar el orden de los números que probamos, obtenemos Sudokus completamente diferentes cada vez que ejecutamos el código.

5. resolverSudoku()

Esta es la función que rellena el Sudoku. Su pensamiento es más o menos: *"Tengo una casilla vacía. Voy a probar números hasta encontrar uno que sirva."*

Por ejemplo, dada la fila: `| 5 3 . |`

- Prueba el 1: ❌
- Prueba el 2: ❌
- Prueba el 3: ❌
- Prueba el 4: ✅ (Entonces pone `5 3 4` y avanza)

¿Y si se equivoca más adelante?
Supongamos que puso `5 3 4 ...` y siguió avanzando. Después descubre que ❌ **No puede completar el resto del tablero**. Entonces hace:

```
Volver atrás ↓ Sacar el 4 ↓ Probar otro número
```

6. generarSudokuCompleto()

Ahora juntamos las piezas básicas para crear un tablero completamente resuelto:

```
function generarSudokuCompleto() { const tablero = crearTableroVacio(); resolverSudoku(tablero); return tablero; }
```

El flujo es simple: **Tablero vacío** → **resolverSudoku()** → **Tablero completamente lleno**. Tenemos una solución válida completa, por ejemplo:

```
8 2 7 | 1 5 4 | 3 6 9 9 4 3 | 6 2 7 | 1 8 5 1 5 6 | 9 3 8 | 7 2 4 ...
```

7. quitarNumeros()

Ahora viene la parte de convertir ese tablero lleno en un rompecabezas de Sudoku listo para jugar. Elegimos algunas casillas al azar y borramos sus números:

Completo

```
8 2 7 | 1 5 4 | 3 6 9 9 4 3 | 6 2 7 | 1 8 5 1 5 6 | 9 3 8 | 7 2 4
```

→

Para jugar

```
8 . 7 | 1 . 4 | . 6 9 . 4 3 | . 2 . | 1 8 . 1 . 6 | 9 . 8 | . 2 4
```

Los números que quedan visibles son las pistas iniciales del jugador.

8. contarSoluciones()

Esta función evita que fabriquemos un Sudoku "roto" (con múltiples soluciones posibles). Cada vez que quitamos un número, el sistema pregunta: *"Si dejo esta casilla vacía, ¿cuántas maneras diferentes existen de completar todo el Sudoku?"*

- **0** → Imposible ❌
- **1** → Una sola respuesta ✅ (¡Esto es lo que queremos!)
- **2+** → Hay varias respuestas ❌

El proceso de validación funciona así:

```
Quito número ↓ Cuento soluciones ↓ ¿1? / \ SÍ NO ↓ ↓ Dejar Volver a ponerlo
```

9. Todo junto: generarSudoku()

Ahora podemos entender la función principal que coordina todo el proceso:

```
function generarSudoku(cantidad) { const tablero = crearTableroVacio(); resolverSudoku(tablero); quitarNumeros(tablero, cantidad); return tablero; }
```

Mentalmente, el algoritmo sigue estos cuatro pasos secuenciales:

🟦 **PASO 1** Crear un tablero vacío.

🟩 **PASO 2** Llenarlo correctamente (tablero resuelto).

🟨 **PASO 3** Quitar números gradualmente.

🟥 **PASO 4** Cada vez que quitamos uno, comprobar: *"¿Todavía tiene una única solución?"* Si no, se restaura.

🎯 RESULTADO: Un Sudoku válido para jugar.

🧠 El mapa que deberías recordar

No hace falta que memorices todavía todo el código. Quedate con este flujo principal:

`crearTableroVacio()` → `resolverSudoku()` → `quitarNumeros()` → `contarSoluciones()` → `Sudoku listo`

Resumen de Responsabilidades

Función	¿Qué hace?
`crearTableroVacio()`	Crea las 81 casillas vacías (matriz de ceros).
`encontrarVacio()`	Busca la próxima posición donde falta un número.
`esValido()`	Comprueba si un número puede ir en una casilla sin romper las reglas.